

Ввод шестнадцатеричного числа через Debug.

Учебно-тренировочная работа.

Написание процедуры Disp_line, выполняющей вывод на экран содержимого шестнадцати байтов. Процедура записана в файл Disp_sec.asm.

```
org 100h
jmp Disp_line
; Данные программы
Sector db 10h, 11h, 12h, 13h, 14h, 15h, 16h, 17h ; Образец
        db 18h, 19h, 1Ah, 1Bh, 1Ch, 1Dh, 1Eh, 1Fh ; текста
; -----
; Процедура дампирует 16 байт памяти в одну строку
; шестнадцатеричных чисел
; -----
; Вызовы: Write_byte_hex, Write_char
; Чтение: Sector
;
Disp_line:
    xor bx, bx          ; Обнуление BX
    mov cx, 16          ; Счетчик байтов
.M: mov dl, [Sector + bx] ; Получить один байт
    call Write_byte_hex  ; Вывод шестн. числа
    mov DL, ' '          ; Вывод на экран
    call Write_char      ; пробела
    inc bx              ; Возврат за следующим
    loop .M             ; байтом
    int 20h             ; Возврат в DOS
    %include 'Video_io.asm' ; Подсоединение процедур
```

Создание com-файла и проверка работы программы

```
C:\PROGRA~1\NASM>nasm Disp_sec.asm -o Disp_sec.com
C:\PROGRA~1\NASM>Disp_sec.com
10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
```

Написание программы, которая осуществляет дампирование 256 байтов памяти

Для создания такой программы Файл Disp_sec.asm был откорректирован и в него добавлена процедура Disp_sector, которая и осуществляет этот вывод

```

org 100h
jmp Disp_sector
; Данные программы
Sector times 16 db 10h ; Первая строка байтов
      times 16 db 11h
      times 16 db 12h
      times 16 db 13h
      times 16 db 14h
      times 16 db 15h
      times 16 db 16h
      times 16 db 17h
      times 16 db 18h
      times 16 db 19h
      times 16 db 1Ah
      times 16 db 1Bh
      times 16 db 1Ch
      times 16 db 1Dh
      times 16 db 1Eh
      times 16 db 1Fh ; Последняя строка байтов
; -----
;
;      Отображает на экран сектор (256 байт)
; -----
; Вызовы: Disp_line, Send_crlf
;
Disp_sector:
    xor dx,dx      ; Начало Sector
    mov cx,16      ; Число строк 16
.M: call Disp_line ; Вывод строки
    call Send_crlf ; Перевод строки
    add dx,16      ; Номер следующего байта
    loop .M        ; Проверка числа строк
    int 20h        ; Выход в DOS
;

```

Также была откорректирована ранее написанная вспомогательная процедура Disp_line, которая находится в том же файле

```

; -----
;      Процедура дампирует 16 байт памяти в одну строку
;      шестнадцатеричных чисел
; -----
; Входы: DX - номер первого байта строки в Sector
; Вызовы : Write_char, Write_hex
; Читается : Sector
;
Disp_line:
    push bx
    push cx
    push dx
    mov bx,dx          ; В BX номер первого байта
    mov cx, 16         ; Счетчик байтов
.M: mov dl, [Sector + bx] ; Получить один байт
    call Write_byte_hex ; Вывод шестнад. числа
    mov dl, ' '        ; Вывод на экран
    call Write_char    ; пробела
    inc bx             ; Возврат за
    loop .M            ; следующим байтом
    pop dx
    pop cx
    pop bx
    ret
%include 'Video_io.asm' ; Подсоединение процедур
%include 'Cursor.asm'  ; --//--

```

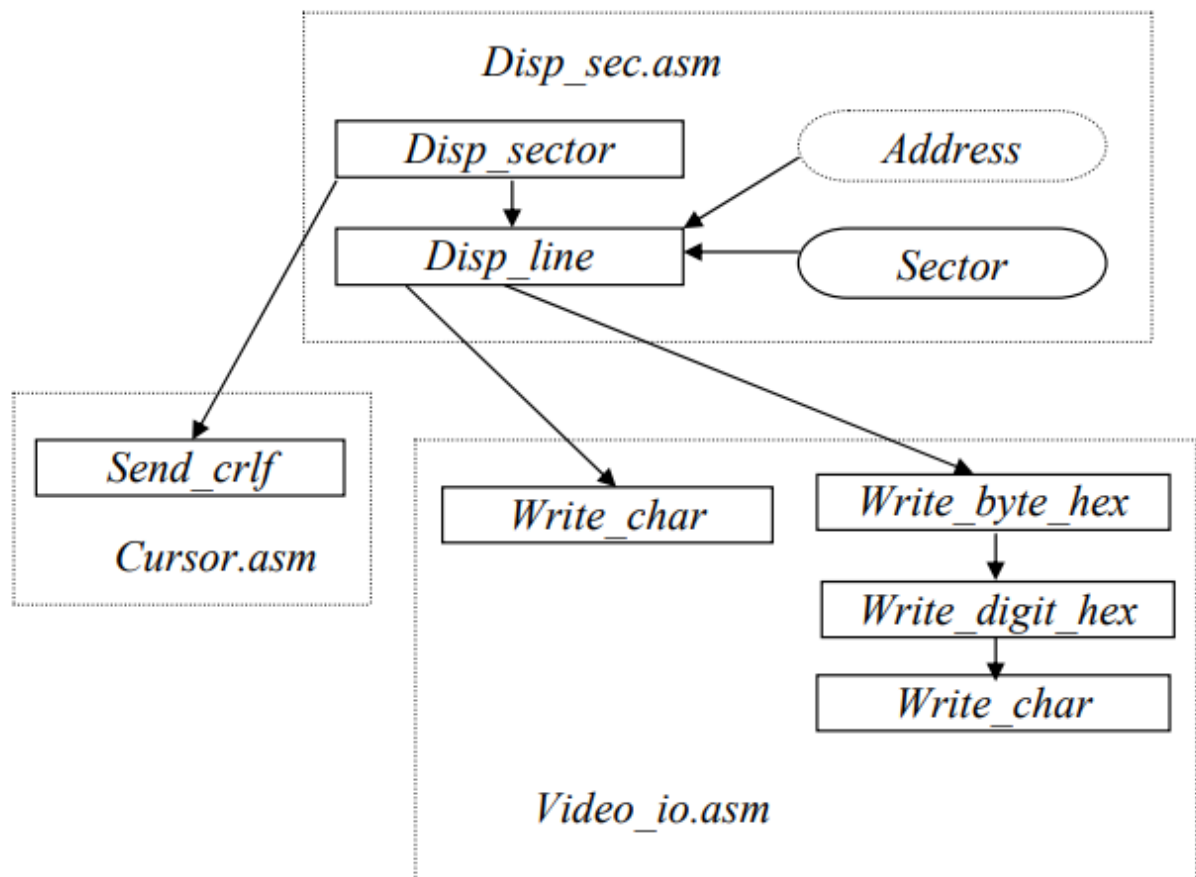
И также добавлена вспомогательная процедура Send_crlf (crlf означает "Carriage Return – Line Feed." – "Возврат каретки – Перевод строки"), которая осуществляет переход на следующую строку, она находится в файле Cursor.asm

```

cr equ 13 ; Возврат каретки
lf equ 10 ; Перевод строки
;
;      Перевод строки экрана
; -----
;
Send_crlf:
    push ax
    push dx
    mov ah, 2 ; функция вывода
    mov dl, cr ; Выводимый символ
    int 21h   ; Вывод символа
    mov dl, lf
    int 21h   ; Прерывания для вывода символа
    pop dx
    pop ax
    ret

```

Так выглядит дерево подпрограмм и файловая структура программы вывода на экран сектора:



Создание com-файла и проверка работы программы

```
C:\PROGRA~1\NASM>nasm Disp_sec.asm -o Disp_sec.com
```

```
C:\PROGRA~1\NASM>Disp_sec.com
10 10 10 10 10 10 10 10 10 10 10 10 10 10 10
11 11 11 11 11 11 11 11 11 11 11 11 11 11 11
12 12 12 12 12 12 12 12 12 12 12 12 12 12 12
13 13 13 13 13 13 13 13 13 13 13 13 13 13 13
14 14 14 14 14 14 14 14 14 14 14 14 14 14 14
15 15 15 15 15 15 15 15 15 15 15 15 15 15 15
16 16 16 16 16 16 16 16 16 16 16 16 16 16 16
17 17 17 17 17 17 17 17 17 17 17 17 17 17 17
18 18 18 18 18 18 18 18 18 18 18 18 18 18 18
19 19 19 19 19 19 19 19 19 19 19 19 19 19 19
1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A
1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B
1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C
1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D
1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E
1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F
```

Программа, выполняющая очистку экрана перед выводом на экран сектора

Файл Cursor.asm был дополнен процедурой Clear_screen, которая осуществляет очистку экрана

```

;
;   Очистка экрана
;   -----
;
Clear_screen:
    push ax
    push bx
    push cx
    push dx
    xor al, al ; Очистить все окно
    xor cx, cx ; Верхний левый угол в (0,0)
    mov dh, 24 ; Нижняя строка экрана -24
    mov dl, 79 ; Правая граница в 79 столбце
    mov bh, 7  ; Применить нормальные атрибуты
    mov ah, 6  ; Очистить
    int 10h    ; окно
    pop dx
    pop cx
    pop bx
    pop ax
    ret
;

```

И была написана программа для тестирования процедуры

```

    org 100h
;
;   Тестирование процедуры Clear_screen
;   -----
;
; Вызовы: Clear_screen
;
Test_clear_screen:
    call Clear_screen ; Вывод числа
    int 20h           ; Возврат в DOS
    %include 'Cursor.asm' ; Включение файла Video_io.asm
;

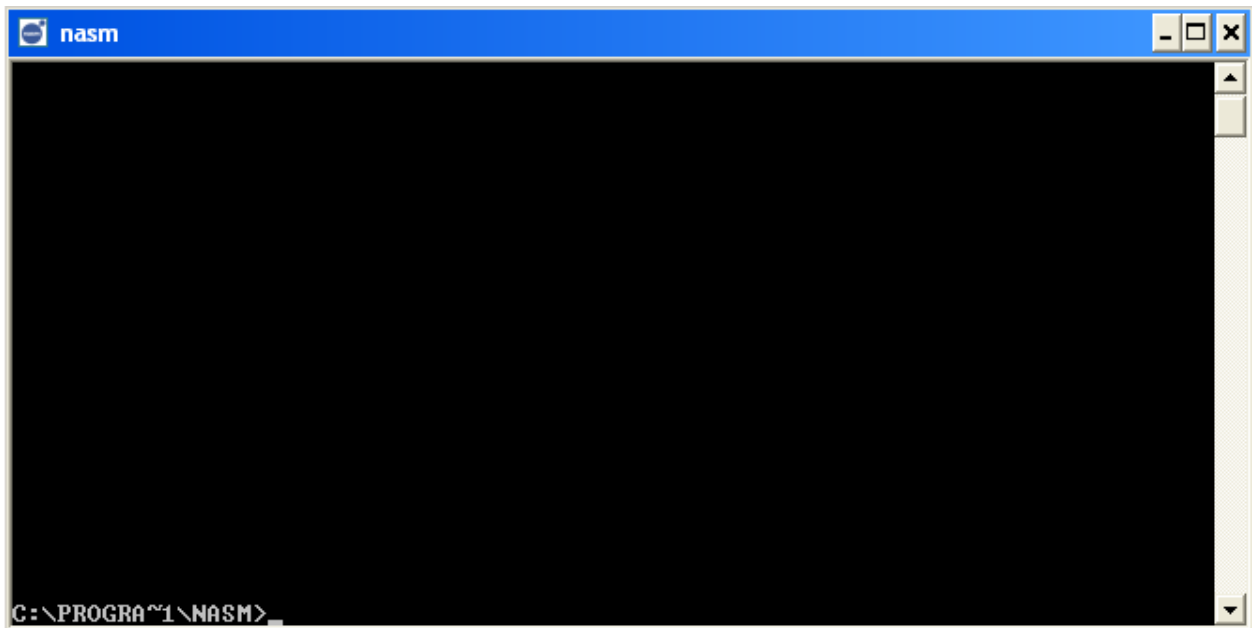
```

Создан com-файл и программа была протестирована

```

C:\PROGRA~1\NASM>nasm Test1.asm -o test.com
C:\PROGRA~1\NASM>test.com_

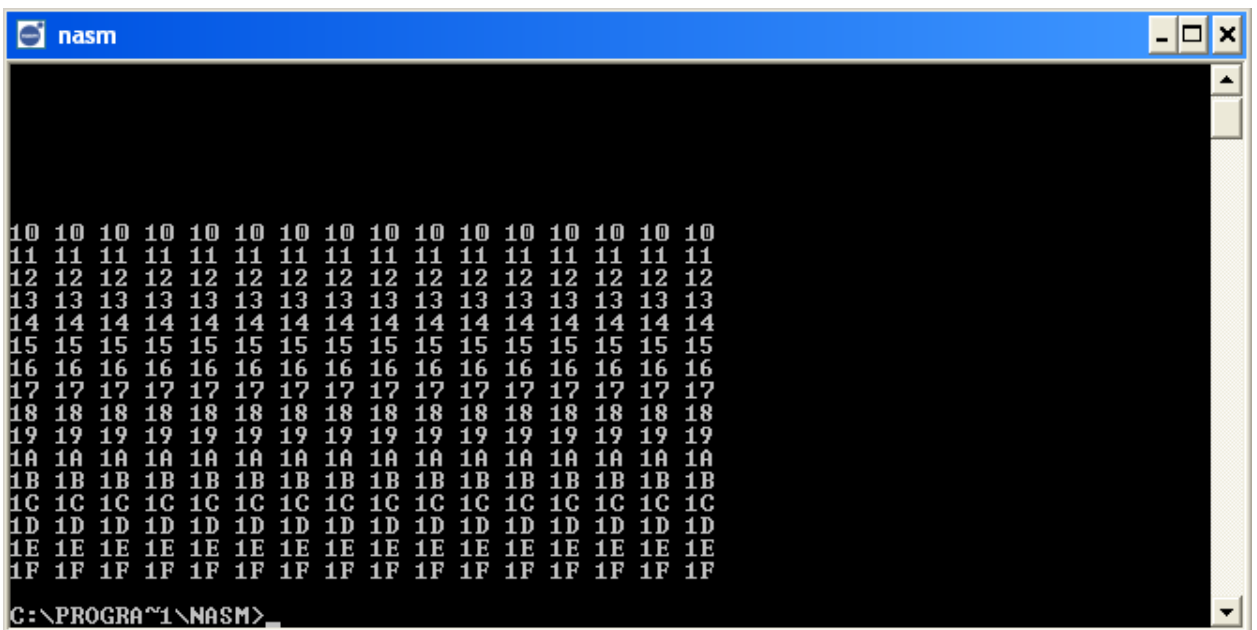
```



Добавление очистки экрана перед выводом сектора

В начало процедуры Disp_sector был записан вызов функции по очистке экрана. Получен com-файл и выведены значения из сектора (проверка работы программы)

```
C:\PROGRA~1\NASM>nasm Disp_sec.asm -o Disp_sec.com
C:\PROGRA~1\NASM>Disp_sec.com
```



Лабораторная работа 10.

Дампирование памяти.

Был откорректирован файл Disp_sec.asm так, чтобы экранная строка дампа содержала бы сначала 4-х позиционный шестнадцатеричный адрес первого байта строки, далее один пробел, после которого шестнадцатеричное представление 16-и байтов памяти. Далее следует еще один пробел, после которого следует символьное представление этих же 16-и байтов (между символами пробелов нет).

```
org 100h
jmp Disp_sector
; Данные программы
Address dw 105h ; Адрес смещение сектора
Sector  db 41h, 42h, 43h, 44h, 45h, 46h, 47h, 00h,
        db 20h, 1Fh, 80h, 81h, 82h, 83h, 84h, 85h ; Первая строка байтов
        times 16 db 11h
        times 16 db 12h
        times 16 db 13h
        times 16 db 14h
        times 16 db 15h
        times 16 db 16h
        times 16 db 17h
        times 16 db 18h
        times 16 db 19h
        times 16 db 1Ah
        times 16 db 1Bh
        times 16 db 1Ch
        times 16 db 1Dh
        times 16 db 1Eh
        times 16 db 1Fh ; Последняя строка байтов
; -----
;
;      Отображает на экран сектор (256 байт)
; -----
; Вызовы: Clear_screen, Disp_line, Send_crlf
;
Disp_sector:
    call Clear_screen ; Очистка экрана
    xor dx,dx         ; Начало Sector
    mov cx,16         ; Число строк 16
.M: call Disp_line    ; Вывод строки
    call Send_crlf    ; Перевод строки
    add dx,16         ; Номер следующего байта
    loop .M           ; Проверка числа строк
    int 20h           ; Выход в DOS
;
```

```

; -----
; Процедура дампирует 16 байт памяти в одну строку
; шестнадцатеричных чисел
; -----
; Входы: DX - номер первого байта строки в Sector
; Вызовы : Write_word_hex2, Write_char, Write_byte_hex
; Читается : Sector, Address
;
Disp_line:
    push bx
    push cx
    push dx
    mov bx,dx                ; В BX номер первого байта
;
; Отображение 4-х позиционного шестнадцатеричного адреса первого байта строки
;
    add dx, [Address]        ; Начальный адрес строки
    call Write_word_hex2     ; Вывод адреса строки
    mov dl, ' '              ; Вывод на экран
    call Write_char          ; пробела
;
; Вывод шестнадцатеричного представления 16-и байтов памяти
;
    mov cx, 16               ; Счетчик байтов
    push bx                  ; Сохранение bx в стеке для последующего вывода символов
.M: mov dl, [Sector + bx]    ; Получить один байт
    call Write_byte_hex      ; Вывод шестнад. числа
    mov dl, ' '              ; Вывод на экран
    call Write_char          ; пробела
    inc bx                   ; Возврат за
    loop .M                  ; следующим байтом
    pop bx                   ; Возврат значения bx для вывода символов
;
;
; Символьное представление этих же 16-и байтов
;
    mov dl, ' '              ; Вывод на экран
    call Write_char          ; пробела
    mov cx, 16               ; Счетчик символов
.N: mov dl, [Address + bx + 2] ; Получить один символ
    call Write_char          ; Вывод символа
    inc bx                   ; Возврат за
    loop .N                  ; следующим символом

    pop dx
    pop cx
    pop bx
    ret
%include 'Video_io.asm' ; Подсоединение процедур
%include 'Cursor.asm'   ; --/--
;

```

При этом скорректирована процедура Write_char в файле Video_io.asm так, чтобы вместо любого управляющего символа (код ASCII от 00h до 1Fh включительно) выводиться символ “.”.


```

;
; Вывод символа на экран (вместо управляющего символа выводится точка)
; -----
;
; Входы: DL содержит код символа, выводимый на экран
;
Write_char:
    push ax
    push dx
    mov ah, 02h    ; функция вывода символа
    cmp dl, 1Fh    ; Проверка управляющий символ в dl или нет
    ja .output     ; Если символ не управляющий, то он сразу выводится
    mov dl, '.'     ; Если символ управляющий, то он заменяется на точку
.output: int 21h    ; Вывод символа на экран
    pop dx
    pop ax
    ret

```

В качестве примера экранной строки первая строка изменена, а другие остались прежними:

A17F 41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85 ABCDEFG. .АБВГДЕ

```

; Данные программы
Address dw 105h ; Адрес смещение сектора
Sector  db 41h, 42h, 43h, 44h, 45h, 46h, 47h, 00h,
        db 20h, 1Fh, 80h, 81h, 82h, 83h, 84h, 85h ; Первая строка байтов
        times 16 db 11h
        times 16 db 12h
        times 16 db 13h
        times 16 db 14h
        times 16 db 15h
        times 16 db 16h
        times 16 db 17h
        times 16 db 18h
        times 16 db 19h
        times 16 db 1Ah
        times 16 db 1Bh
        times 16 db 1Ch
        times 16 db 1Dh
        times 16 db 1Eh
        times 16 db 1Fh ; Последняя строка байтов

```

Создание com-файла и проверка работы программы

```

C:\PROGRA~1\NASM>nasm Disp_sec.asm -o Disp_sec.com
C:\PROGRA~1\NASM>Disp_sec.com_

```

```

0105 41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85  ABCDEFG. .АБВГДЕ
0115 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11 11  .....
0125 12 12 12 12 12 12 12 12 12 12 12 12 12 12 12 12  .....
0135 13 13 13 13 13 13 13 13 13 13 13 13 13 13 13 13  .....
0145 14 14 14 14 14 14 14 14 14 14 14 14 14 14 14 14  .....
0155 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15 15  .....
0165 16 16 16 16 16 16 16 16 16 16 16 16 16 16 16 16  .....
0175 17 17 17 17 17 17 17 17 17 17 17 17 17 17 17 17  .....
0185 18 18 18 18 18 18 18 18 18 18 18 18 18 18 18 18  .....
0195 19 19 19 19 19 19 19 19 19 19 19 19 19 19 19 19  .....
01A5 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A 1A  .....
01B5 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B 1B  .....
01C5 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C 1C  .....
01D5 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D 1D  .....
01E5 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E 1E  .....
01F5 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F 1F  .....

```

```

C:\PROGRA~1\NASM>

```